



WP Suomi 2026, Oulu, 16th of October, 2026

Going *ACF-free*

Replacing a plugin dependency
with core WordPress

Rolle Laukkarinen

Founder and CTO, Digitoimisto Dude Oy

       → rolle.social



About *Rolle*

Founder and CTO of Digitoimisto Dude Oy. Code soon 30 years, WordPress since 2005, online since 1999. Accessibility, open source, Linux servers, and building my own tools. Free time: running, bilingual family, knows sign language.

Dude & *WordPress*

Contributing to WordPress and open source since we founded Dude in 2013: 71 open source repositories on GitHub, including air-light, our starter theme with over 1100 stars.



Full stack

Front end

Theme dev

AI

Accessibility

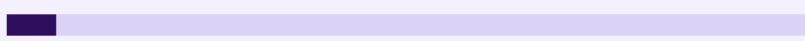
Design

Business

Automation

Open source

Self-hosting



No shame in *using ACF or plugins*

WordPress has a great plugin ecosystem, and ACF is a very good plugin.
ACF built a lot of good sites, ours included.

A quick *poll*

How many of you...

A quick *poll*

How many of you...

- 1 Still use ACF for everything (okay, with exceptions, let's say 80%)

A quick *poll*

How many of you...

- 1 Still use ACF for everything (okay, with exceptions, let's say 80%)
- 2 Use ACF for some things

A quick *poll*

How many of you...

- 1 Still use ACF for everything (okay, with exceptions, let's say 80%)
- 2 Use ACF for some things
- 3 Have fully moved on to native, React or core Gutenberg blocks

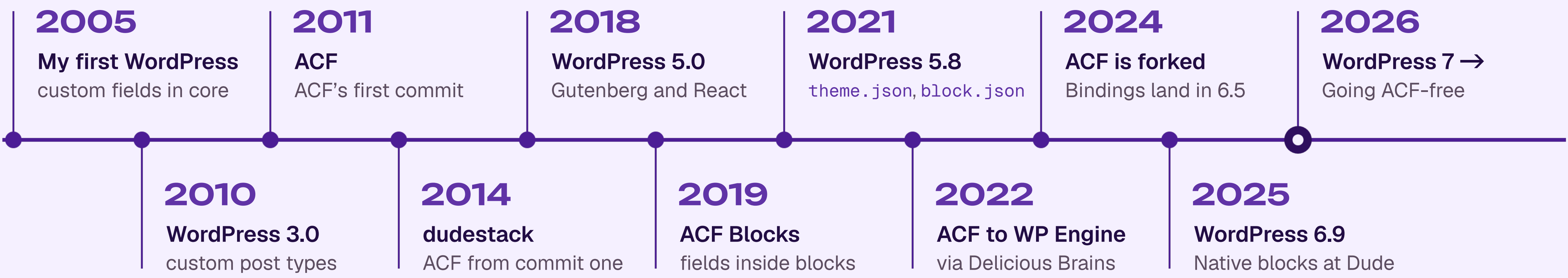
What is *a dependency?*

“A situation in which you need something or someone and are unable to continue normally without them.”

Cambridge Dictionary

A brief history of *fields and dependencies in WordPress*

Fields and blocks *since 2005*



~~Advanced~~ Custom fields

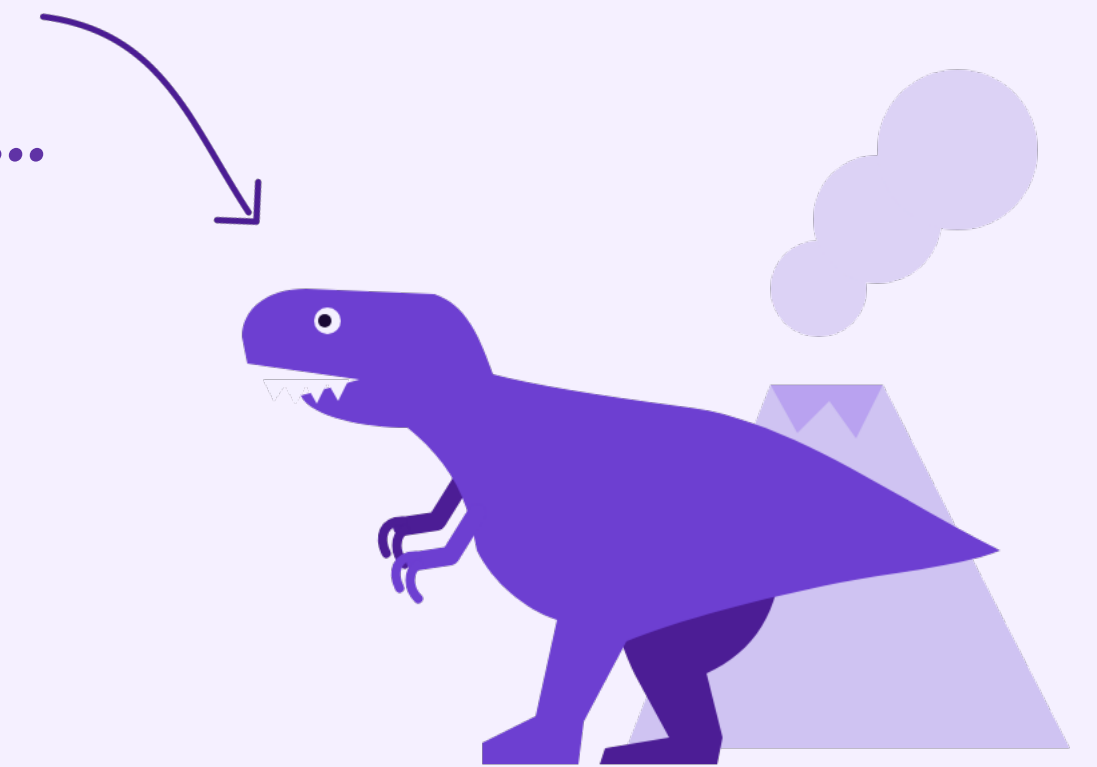
What is a field, anyway?

We are talking about fields, right?

No dependencies?

Let's knife the database!

*Let's go back to
prehistoric times...*



- In the early days, some of us used plain meta fields, and it was enough.
- For a developer, `get_post_meta()` is fine. It is a knife straight into a row.
- Once clients edit content themselves, they need a proper editing interface. Also, text fields are way too limited.

Custom Fields

^

v

▲

Add New Custom Field:

Name	Value
favorite_food	burrito

[Cancel](#)

[Add Custom Field](#)

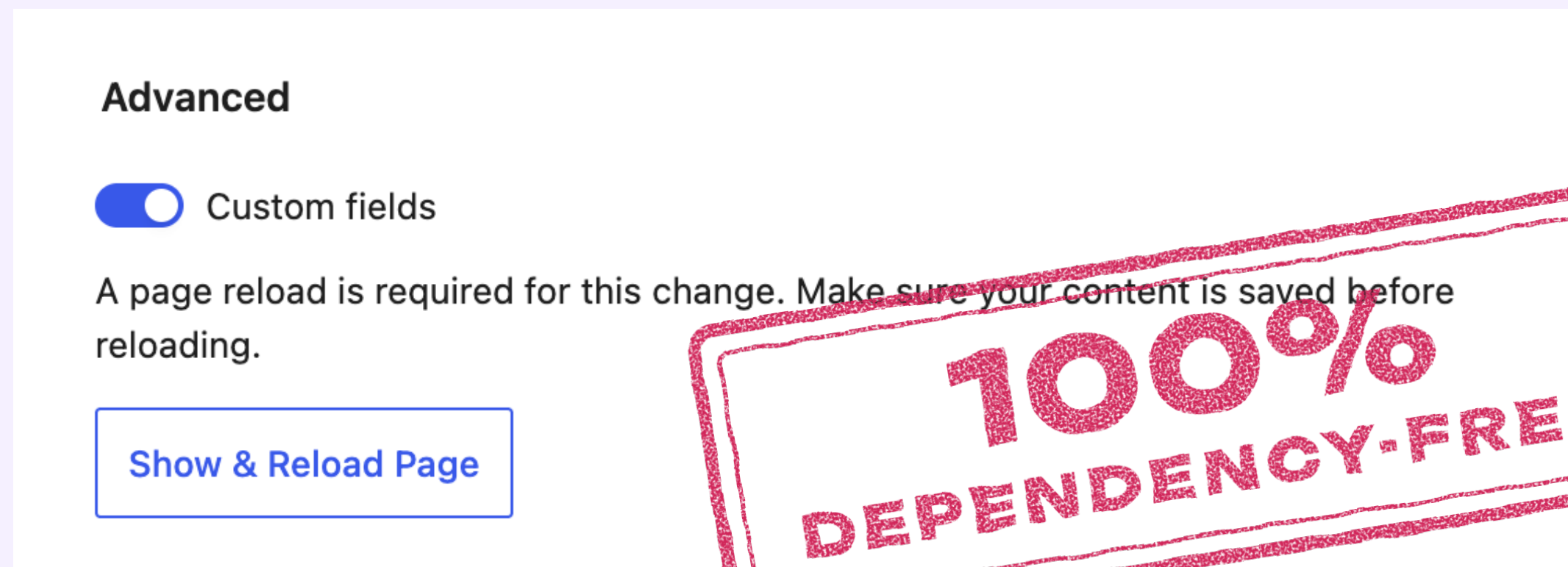
BEST BEFORE

2009

How to use *(prehistoric)* custom fields

Turn them on

Options (three dots), Preferences, General, Advanced, Custom fields. The editor reloads.



ACF actually disables them

ACF removes the Custom Fields box by default, so on an ACF site the toggle is not there at all. One filter brings it back.

```
add_filter(  
    'acf/settings/remove_wp_meta_box',  
    '__return_false'  
);
```

Hidden in Preferences since 2018. Bring a shovel.

Advanced Custom Fields

since 2011

ACF was born to extend custom fields.

Better fields, but still fields.

We are past the field era and into the block era.

We thought we did “*the core way*”

Underscores, 2016

air-light started from Automattic’s starter theme, a traditional theme kept as close to core as we could.

ACF on every site

The content in those templates came from ACF fields. That is not the core way.

Core functions

Templates built from WordPress’s own parts: the loop, template parts, `WP_Query`.

Why we kept doing ACF

ACF was quick to build with, and the sites built on it kept working for years.

So what's *the answer?*

When we finally went native in 2025,
we thought we knew.



We thought “native” *meant React.js*

2018 to 2024

We used core Gutenberg for articles,
ACF for everything else.
ACF Gutenberg blocks since 2021.

One site, when we still stumbled

23 custom blocks, 77 attributes, about
3000 lines of edit.js JavaScript.

2025

Our first real native blocks:
every option written by hand in React.

The result

Too static blocks, “reinventing the
wheel” like with ACF...



And suddenly we were React.js – *devs*

2012

Underscores. PHP, HTML and CSS. Where I learned themes, and where I stayed for the longest time.

2017

WordPress nearly dropped React over its patent clause. Facebook relicensed it as MIT.

Flashback, historical hard pass

Sage, Twig, Timber, Blade, Dust.js, exotic ways of building WordPress. I kept at doing traditional WordPress.



2018

Blocks are React components, built with npm and [@wordpress/scripts](#).

Media and text, *coded by hand*

NOT "QUITE"
THE CORE WAY

```
blocks/media-text/  
├─ block.json  
├─ build/  
└─ src/  
    ├─ index.js  
    ├─ edit.js  
    ├─ save.js  
    ├─ deprecated.js  
    ├─ editor.scss  
    └─ style.scss
```

```
const { heading, level, mediaUrl,  
      mediaRight, bgColor, padding } = attributes;  
<InspectorControls>  
  <SelectControl label="Heading level" value={ level } />  
  <ToggleControl label="Media right" checked={ mediaRight } />  
  <ColorPalette value={ bgColor } onChange={ setBg } />  
  <RangeControl label="Padding" value={ padding } />  
</InspectorControls>  
<RichText tagName={ level } value={ heading } />  
<MediaUpload onSelect={ setMedia } render={ button } />
```

Every one of these controls already ships with core Media & Text, in core since WordPress 5.0 (2018).

React is for *WordPress core developers*

The WordPress block editor itself is built with React.js.

Themes, patterns, styles and theme.json need none of it.

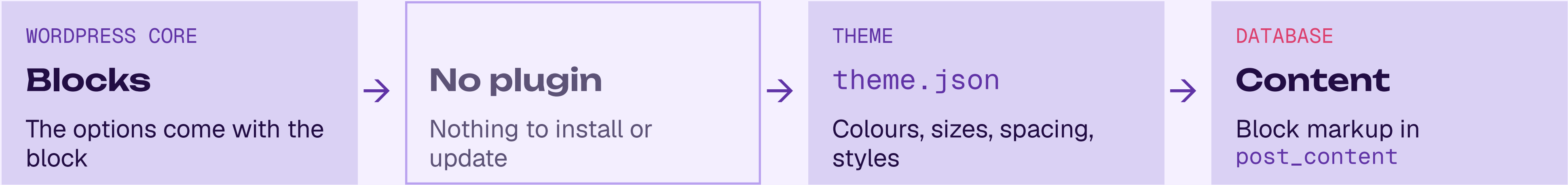
React only comes in for a custom block, and that's the last resort.

Where your content *lives*

With ACF



With core blocks



True dependency-free WordPress *is the core way*

The WordPress block editor (also called simply the block editor, WordPress editor or Gutenberg editor) as the content management system, with no plugin that the content depends on.

Three ways to *build a sovereign theme*

Traditional

CORE WAY

```
_s/
├── style.css
├── functions.php
├── header.php
├── footer.php
├── index.php
├── single.php
├── page.php
├── template-parts/
│   └── content.php
└── inc/
```

Underscores, 2012. PHP templates and the template hierarchy.

Innovative

NOT
CORE WAY

```
sage/
├── app/
│   └── View/Composers/
├── resources/
│   ├── views/
│   │   └── index.blade.php
│   ├── css/
│   └── js/
├── composer.json
├── package.json
└── vite.config.js
```

Sage by Roots. Blade, Composer and a build step on top of WordPress.

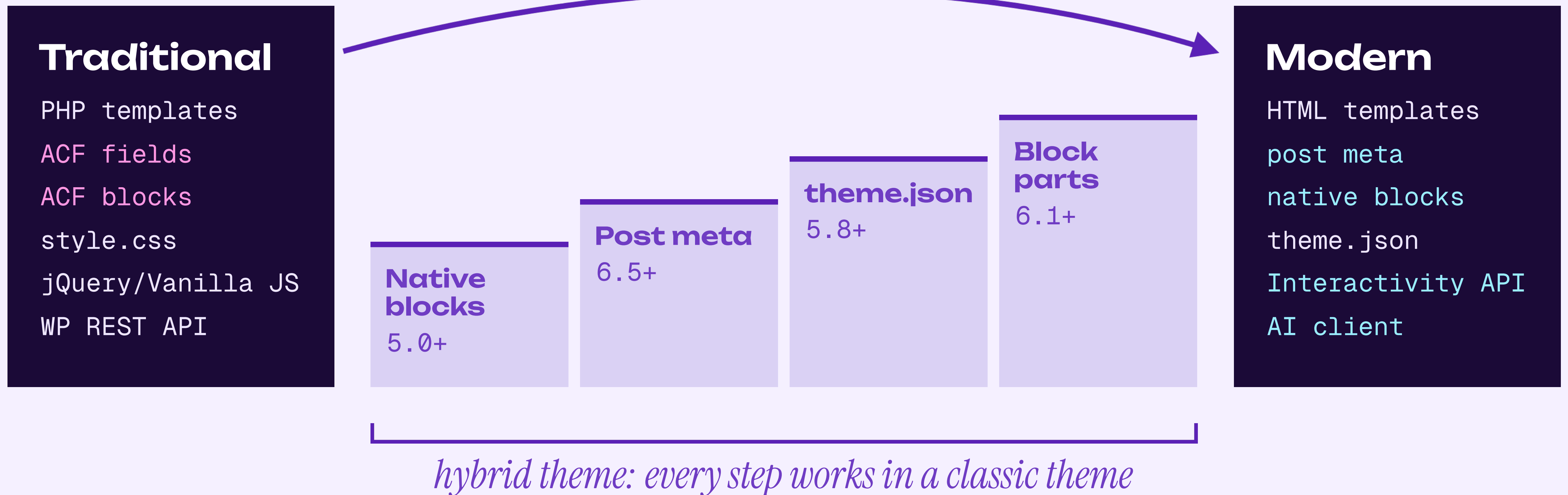
Modern

CORE WAY

```
block-theme/
├── style.css
├── theme.json
├── functions.php
├── templates/
│   ├── index.html
│   └── single.html
├── parts/
│   └── header.html
├── patterns/
└── styles/
```

Block theme, WordPress 5.9. HTML templates and `theme.json`.

From Traditional *to Modern*



Why go *100% core*

Pros

- Performance, things moving from server to browser
- Heading levels, colours and spacing are built in.
- The client edits in the preview itself.
- Lego and Duplo at once, mixed freely.
- Anything can be moved, reordered and combined.
- The client gets a real tool. One client had called an old site “the WordPress straitjacket Dude knitted”.

“Cons”

- A steep learning curve for the whole team.
- Less PHP, more JSON: `theme.json`, `block.json`.
- Unlearning building every block from scratch.
- Heavy build tooling: React, npm, a build step.
- Core moves fast, and the docs are scattered.



The designer's *new job*



We used to design the limits: fixed layouts the client could only fill in.

Core gives the client every block, in any order, next to anything.

Every block needs to look like the brand, in every combination.

The leap is high, *and that is not your fault*

Since WordPress 5.0 in 2018, Gutenberg has added many ways to build sites without dependencies. And it has been a ride.

Ways to build *a block*

- Core blocks, as shipped
- Core block variations
- Block patterns
- Block styles
- Filtering a core block in PHP
- Filtering a core block in JS
- Synced patterns
- `register_block_type` in PHP
- `registerBlockType` in JS
- `block.json` metadata
- Dynamic block with `render.php`
- Custom React block
- Custom block with `InnerBlocks`
- Interactive block, Interactivity API
- `acf_register_block_type`
- ACF blocks via `block.json`
- Hybrid ACF and native
- Classic theme, core blocks
- Block theme, full site editing
- Block Bindings
- 3rd party page builders

From core *to custom, prioritized*

1. Core blocks

WordPress 7.1 ships 115 of them, including accordion, tabs and details.

2. Patterns

Core blocks arranged into a layout that editors insert and edit freely.

3. Variations and styles

A core block with your own defaults or look, in a few lines of code.

4. Custom blocks

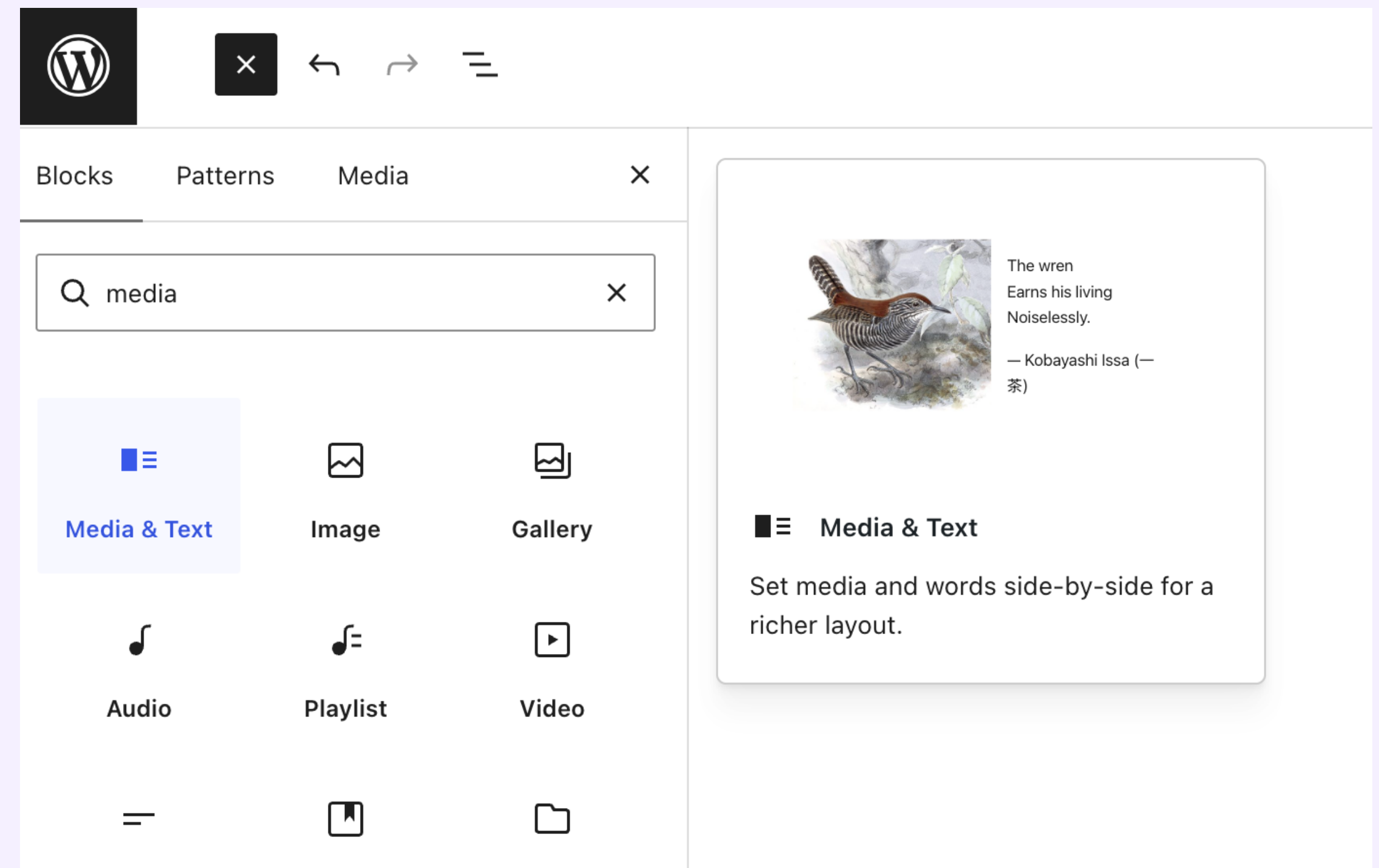
`block.json` and `render.php` when nothing in core fits, with `InnerBlocks` inside.

Block example: Media and text

With ACF

```
$title = get_field( 'title' );
$content = get_field( 'content' );
$image = get_field( 'image' );
$link = get_field( 'link' );
?>
<section class="block block-content-image">
  <h2><?php echo esc_html( $title ); ?></h2>
  <?php echo wp_kses_post( $content ); ?>
  <p>
    <a href="<?php echo esc_url( $link['url'] ); ?>">
      <?php echo esc_html( $link['title'] ); ?>
    </a>
  </p>
  <?php native_lazyload_tag( $image ); ?>
</section>
```

With the core Media & Text block



Most of the time *you don't need to code a block*

WordPress 7.1 ships 115 core blocks,
and most layouts are a core block, a pattern or a block style,
set up in the editor and in `theme.json`.

When you do code: *the Dude House Way*

- 1 Lock `theme.json` first: palette, type scale, spacing...
- 2 Core blocks, curated per post type.
- 3 Style core blocks and their variations in `theme.json`
- 4 Layouts as patterns, built from core blocks.
- 5 Public data from the WP REST API, rendered in the browser.
- 6 Interactivity API for filters and paging.
- 7 Custom blocks only as the last resort.

```
blocks/events/  
├─ block.json  
├─ build/  
└─ src/  
    ├─ index.js  
    ├─ edit.js  
    ├─ view.js  
    └─ style.scss
```

Site Editing/Block theme: *our first build*

Templates and parts

Page templates, header and footer as HTML files, declared in `theme.json`.

Patterns in files

Reusable sections as pattern files, versioned in git with the theme.

The catch

Site Editor changes are saved to the database and quietly override the files.

Where core ran out

Core Navigation as a sidebar menu needed custom JavaScript.

Fields outside blocks, *without ACF*

Custom post type

`register_post_type()`, in the theme. CPTs never needed plugins or dependencies. As a dev, please do not use CPT UI or similar.

Custom plugin, API or MCP

Replacing a dependency with a better one?
It's often worth it.

Fields

`register_post_meta()` with the Block Bindings API to connect a core block to a field.

Full Site Editing / Block theme

Works great, especially for static marketing sites.

This is a lot,
where to start, going 100% ACF-free?

It has always been *a pattern*

Most blocks could be patterns

If an element only consists of static text, images and buttons, it can be a pattern.

Change once, everywhere

Edit a synced pattern and every page that uses it updates.

Nothing to build

Arrange core blocks once, then save them as a pattern.

In core since 2020

Patterns since WordPress 5.5, synced patterns since 6.3.



Start every page *from a pattern*

With patterns in theme files

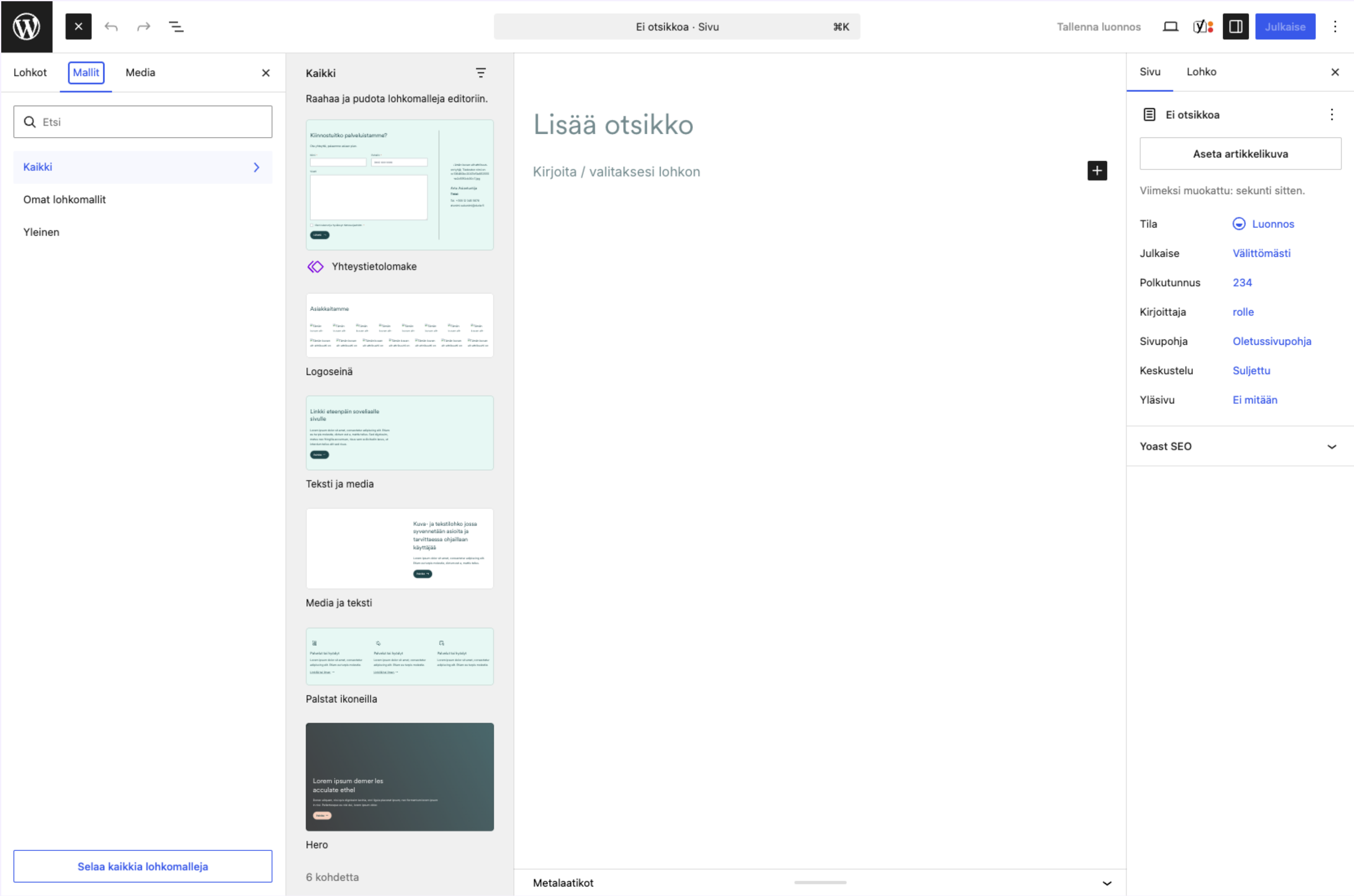
```
register_block_pattern( 'mytheme/landing', [  
    'title'      => 'Landing page',  
    'blockTypes' => [ 'core/post-content' ],  
    'content'    => $markup,  
] );
```

With patterns saved in the editor

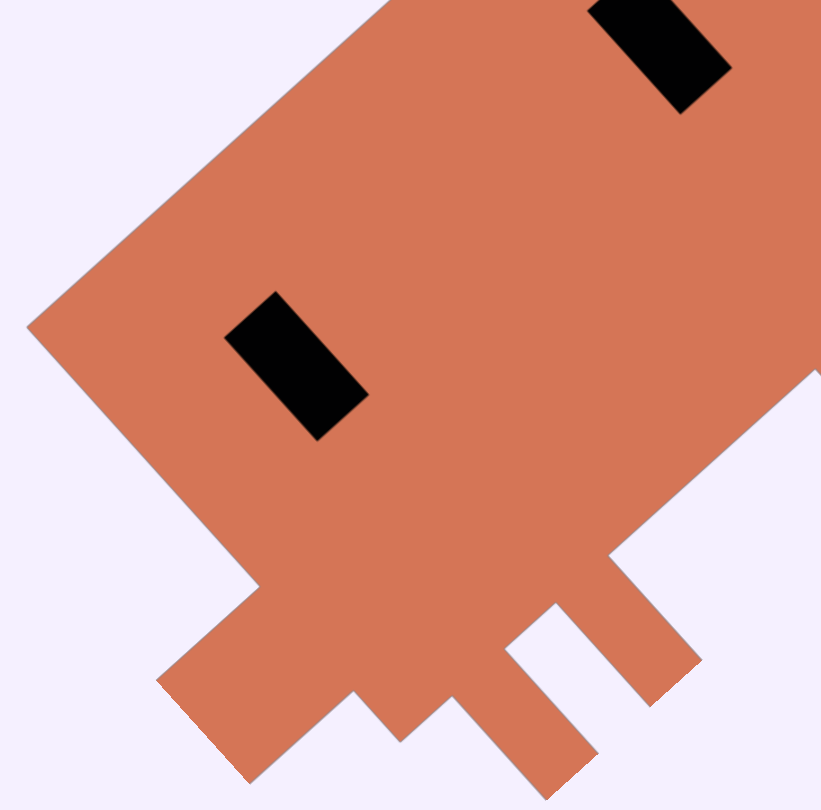
```
dispatch( 'core/editor' ).setIsInserterOpened( {  
    tab: 'patterns',  
    category: {  
        name: 'allPatterns',  
        label: _x( 'All', 'patterns' ),  
    },  
} );
```

On a new empty page, the editor opens straight on the pattern library.

What does it look like?



Learning this *with your AI*



- There are many ways to build a block and the docs are scattered, so AI gets it wrong too.
- AI makes mistakes. It often writes a React.js block, because that is what the official Block Editor Handbook “Build your first block” tutorial with [@wordpress/create-block](https://twitter.com/wordpress/create-block) teaches.
- Tell it the order: core blocks, patterns, styles, and a custom block last.
- Make it show the handbook page it used, learn, review, check the code before you ship it.

When ACF **still** *makes sense*

Settings pages

With ACF, a page for site-wide settings takes a few lines.
Without it, you code the whole page.

Relations

Linking posts together: ACF has a ready-made picker. With core it means coding it by hand.

Advanced fields

Literally what the plugin was made for. If you need a field fast, ACF provides that for you.

Where to *read next*

fullsiteediting.com

Carolina Nymark's lessons on blocks and block themes.

developer.wordpress.org/news

The WordPress Developer Blog.

learn.wordpress.org

Free courses and lessons.

gutenberg.10up.com

10Up's Block Editor Best Practices.

developer.wordpress.org/block-editor

The Block Editor Handbook.

github.com/digitoimistodude/air-light

Our ACF-free WordPress starter theme for devs.



Kiitos. *Questions?*

Rolle Laukkarinen

rolle.social · dude.fi · github.com/rollecode

Slides: github.com/rollecode/wpsuomi2026-talk-slides

